

Developing Plugins Using the Cabbage Framework as a Means to Complement A Composer's and Sound Designer's Toolset And Issues For Distributing the Results in Online Stores

AuthorA¹ and AuthorB² *

¹ InstituteA

² InstituteB

icsc2022.music@gmail.com

Abstract. Since first learning about Csound in 2020, I have distributed four VST and AU plugins developed with the Cabbage framework. Those plugins were developed as a way to enhance my own toolset as a composer and sound designer, with also the aim of sharing those tools over the internet in an accessible manner.

This paper presents an overview of three of these plugins along with a discussion of how the Cabbage framework can be leveraged by composers and sound designers, even those new to programming, to create synthesis and DSP plugins to both fill gaps in their own toolkits, as it did for me, and distribute them to the benefit of the audio community at large.

Keywords: Csound, Cabbage, Plugins, Distribution

1 Introduction

The Cabbage framework provides a powerful environment for developing audio plugins with Csound. In this paper I will provide an account of my experience developing synthesizers and DSP plugins with it, encompassing how I started learning Csound, an overview and dissection of the plugins I developed and why I developed them, and the issues and successes I have found with distributing them through the *itch.io* online storefront.

2 Overview of the Plugins I Developed

The three plugins I will talk about on this paper are *Vendaval*, a synthesizer that models the sound of wind, *Granulera*, a synthesizer designed to produce aleatoric granular textures, and *Cristalera*, a granular glitch DSP plugin.

* Please place acknowledgment here.

Vendaval was originally conceived for a video game I was working on as a sound designer. The idea was to create a procedural synthesis algorithm with Csound that would be rendered in real-time by Unreal Engine as a FMOD plugin compiled with Cabbage. After writing the Csound code for it, I realized it could be a useful tool for sound design outside of that specific project if I transformed it into a MIDI synthesizer, which Cabbage made it easy to do.

Granulera, like *Vendaval*, was also created out of necessity for a project. I was hired to write the soundtrack for the game *Apotheosis* and wanted to incorporate granular textures into the score but didn't have access to any granular synthesizers or DSP effects at the time, so I decided to develop my own synthesizer to fulfill that need.

Cristalera is essentially the DSP version of *Granulera*. I was producing a song in collaboration with a friend and wanted the background vocals to have the same sort of aleatoric granular texture I achieved with *Granulera*. *Cristalera* employs the same granulation method, but using it to process live audio input instead of a synthesized signal.

3 Teaching Myself Csound From Scratch

The ability that Cabbage offers to create my own tools for audio manipulation was the main force that drove me to learn Csound. I had almost no programming knowledge or experience before learning Csound, having only started learning C# scripting and written my first line of code only a couple of months before a friend introduced me to Cabbage, but the notion of being able to create my own synthesizers and audio effects suited to my own needs compelled to learn Csound. It is fortunate that the Csound website provides a comprehensive list of resources and documentation for newcomers to get started, with the *Csound FLOSS Manual* being the resource I've found more useful, providing a very thorough guide of the language from the very basics to more advanced applications, with the plethora of examples being very efficient in helping newcomers grasp the language as well as giving a glimpse of the myriad of possibilities that can be done with it.

I started studying Csound by myself around June 2020 and by the end of the year I had already developed three plugins and distributed two of them. Another resource that proved itself fundamental to teaching myself Csound was the Cabbage forum. I feel a deep sense of gratitude to all of the help I've been given there by Rory Walsh and other experienced users whenever I had trouble trying to implement a feature on one of my plugins.

4 Dissection of Each Plugin

4.1 Vendaval

*Vendaval*³ works by generating noise, filtering it with a bandpass, and modulating the filter's center frequency randomly with the jspline opcode. There's

³ *Vendaval* is available for download here: caiojmini.itch.io/vendaval

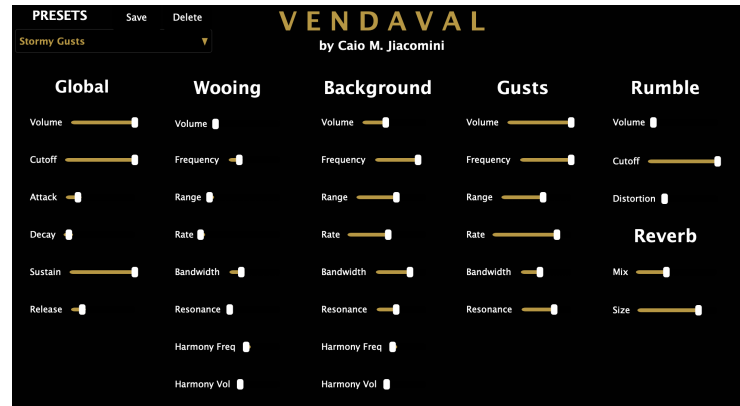


Fig. 1. *Vendaval's* user interface.

also a low-pass filter tuned to the same frequency as the band-pass in order to allow the user to add resonance to the sound. The plugin has four modules that perform different functions employing this general chain: *Wooing*, *Background*, *Gusts*, and *Rumble*.

The *Wooing* and *Background* modules work in the exact same way as described above but offer different value ranges for the user, with *Wooing* having the band-pass filter tuned to a higher frequency range with a narrower bandwidth while the *Background* module has the filter tuned to a lower frequency range with a broader bandwidth. The *Gusts* module applies a LFO to the filter frequency after the spline modulation, as a way to simulate rapid gusts of wind. The final module, *Rumble*, simply applies a low-pass filter to pink noise and distorts it, allowing the cutoff frequency to be set up to 200 Hz, in order to add a present low end for higher intensity wind soundscapes.

In a later update, a secondary band-pass filter that harmonizes with the first one filter was added to the *Wooing* and *Background* modules to simulate the resonance created by wind blowing through small holes. The user has control over the volume of the signal that passes through that filter as well as the frequency of the harmony, given as a multiplier of the base frequency.

The user is given controls over each module's volume, filter frequency, the jspline range and rate, the bandwidth of the band-pass filter, as well as the low-pass resonance.

Filtering pink noise in this fashion is a common technique for synthesizing wind sounds, but wrapping it up in a plugin like this with different modules performing different functions allows the user to create more dynamic and natural sounding wind quicker than if they were to filter pink noise from scratch in a digital audio workstation.

Excerpt of The Code For Vendaval's Background Module

```
aNoise noise 1, 0
```

```

aNoiseBp butterbp aNoise, kCenterFrequency + kJitter, kBandwidth
aNoiseLp moogladder aNoiseBp, kCenterFrequency + kBandwidth + kJitter, kResonance

aNoiseBpHarm butterbp aNoise * kHarmonizerVol, (kCenterFrequency + kJitter) * kHarmonize
aNoiseLpHarm moogladder aNoiseBpHarm, (kCenterFrequency + kBandwidth + kJitter) * kHarmonize
aNoiseBalanced balance aNoiseLp + aNoiseLpHarm, aNoiseBp

aBackground = (aNoiseBalanced * kVolume) * kBackgroundVolume

```

Excerpt of The Code For Vendaval’s Background Module

4.2 Granulera

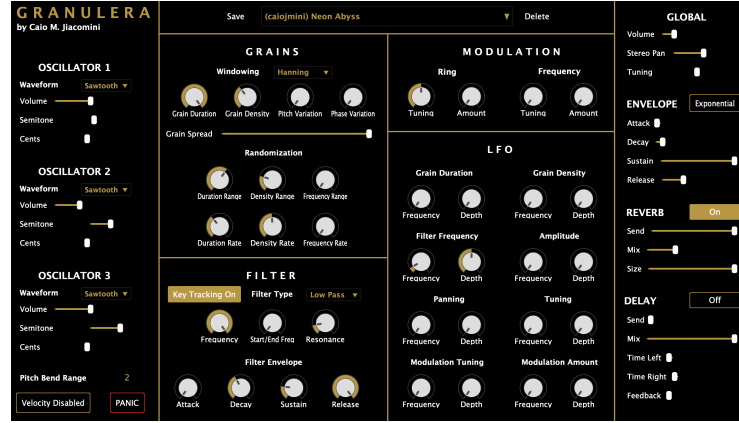


Fig. 2. *Granulera*’s user interface.

*Granulera*⁴ features three oscillators capable of producing basic waveforms; a filter section with the choice of low-pass, high-pass, and band-pass filters; an ADSR envelope for the filter frequency; reverb and stereo delay effects; LFOs for key parameters; an ADSR envelope for the instrument’s global amplitude; ring modulation and, originally, amplitude modulation. With regards to the granulation, it offers control over the windowing shape, grain duration, grain density, as well as control over the pitch and phase randomization range for each grain. On top of it, the grain duration and density values, as well as the global tuning of the instrument, are modulated randomly with the jitter opcode, with the interface allowing control over the range and rate of the modulation.

For the first version of *Granulera*, the granulation was made with the grain3 opcode. This allowed me to quickly develop the plugin in time to be used in the

⁴ *Granulera* is available for download here: caiojmini.itch.io/granulera

game's soundtrack, but one thing I was missing was the ability to randomize the stereo position of each individual grain. In a later update, I refactored the granulation method, now using the `schedkwhen` opcode to trigger each grain individually, allowing panning values to be passed in for each grain as a `p-field`. A slider was added to the interface to allow the user to control how spread out in the stereo field the grains can be. The amplitude modulation was also substituted for frequency modulation on the update.

Surprisingly, the trickiest part of developing *Granulera* was making it so the filter envelope's release behaved in a natural way if the user automated the filter frequency value in the middle of the envelope. The issue was that the release stage of the envelope would always start from the frequency value that was defined during the initialization pass for the note, so if the user automated the filter frequency either through a MIDI CC or an automation lane in a digital audio workstation, the frequency would abruptly jump to whatever value it was set to during i-time for the release stage. The solution was to use the `changed2` opcode to detect if the filter frequency value has been altered and, if it's altered, change the value of another variable, using that value in an if statement to change the value passed to the envelope.

Excerpt of The Code For Granulera's Filter Envelope Solution

```
kFilterChangedTrig changed2 gkFilterFreq
kFilterChangedTrig2 init 0

if kFilterChangedTrig == 1 then
    kFilterChangedTrig2 = 1
endif

if release() == 0 then
    if kFilterChangedTrig2 == 1 then
        kFilterFreqSum = gkFilterFreq
    else
        kFilterFreqSum = kFilterEnv
    endif
else
    kFilterFreqSum = kFilterEnv
endif
```

Excerpt of The Code For Granulera's Filter Envelope Solution

4.3 Cristalera

*Cristalera*⁵ uses the same `schedkwhen` granulation method to process live audio input. This was achieved with four Csound instruments: one that is constantly reading the dry input signal with the `inch` opcode, writing it to a channel with the `chnmix` opcode; one

⁵ *Cristalera* is available for download here: caiojmini.itch.io/cristalera

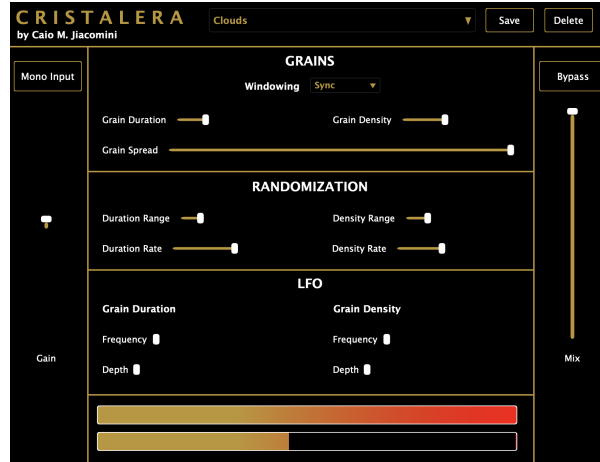


Fig. 3. *Cristalera's* user interface.

instrument that triggers the grains with `schedkwhen`; one instrument that reads the input signal and envelopes it in a window when triggered by the `schedkwhen`, writing the granulated signal to another channel; and one instrument that joins the dry and the affected audio, using the `ntrpol` opcode to mix between both signals.

(add one excerpt of code)

Excerpt of The Code For Cristalera's Granulation Method

```
instr GrainTrigger
//Grain Duration
(...)
    kDurTotal    limit    kDur + kDurVar + kLfoDur , 0.01, 0.9
//Grain Density
(...)
    kDensityTotal    limit    kDensity + kDensityVar + kLfoDensity, 0.1, 80
    kTrigger metro kDensityTotal
    schedkwhen kTrigger, 0, 0, "Grains", 0, kDurTotal
endin

instr Grains
    aSig chnget "DrySignalSum"
    //WINDOWING
        iWfn = chnget:i("WindowingSelection")
        iBalance = 1 - p3
        kWindowIndex line 0, p3, 1
        kWindowEnv table kWindowIndex, iWfn, 1
        aWindowEnv interp kWindowEnv
        aSig *= (aWindowEnv * iBalance)
    (...)
    //OUTPUT
        aGrainL, aGrainR pan2 aSig, .5 + iRandomPan
```

```

    chnmix  aGrainL, "GrainSignalL"
    chnmix  aGrainR, "GrainSignalR"
endin

```

Excerpt of The Code For Cristalera's Granulation Method

5 Distribution: Issues and Succeses

My primary goal with developing these plugins was to expand my toolkit for audio creation and manipulation. A secondary goal was to distribute them online in an accessible manner for the benefit of the audio community at large: if the tools I'm making are useful for me, they most likely are useful for someone else, so their distribution was a concern of mine from the start. I decided to use the *itch.io* storefront to host my work. Despite the focus on being a platform to distribute and sell independent video games, *itch.io* is flexible enough to support the distribution of software of all kind, allowing developers to host their products for free and define themselves the percentage of their sales that goes towards the website. For the sake of accessibility, I opted to distribute my plugins in a pay-what-you-want model with a suggested donation amount, allowing users to download and use them for free, with the option of supporting my work monetarily if they can afford it and think it's valuable to them.

An issue that surfaced with distribution was my lack of resources and knowledge to test the plugins in different versions of operational systems. While most people that downloaded them had no trouble with their setup, I did receive a good number of inquiries from people who had issues running the plugins on both Windows and Mac platforms, reporting problems that I had no idea how to solve. I tried to work with them to the best of my ability but I didn't have the knowledge to help them nor the time to get that knowledge. One person that was having trouble running *Vendaval* on a Macbook told me that it magically started working after reinstalling Csound five times

Another issue, this one particular to the distribution over *itch.io*, was how the platform would sometimes charge duplicate payments from people that chose to donate their money, which led to me having to waste time scrutinizing every transaction to refund accidental duplicated payments. I'm not sure how pervasive this issue is on the platform or if it's tied at all to it being tied to a PayPal account based on my home country.

Although I was unable to directly help most people that had problems running my plugins, the amount of positive responses and interactions I had far overshadowed those issues. *Vendaval* has been my most successful plugin so far, both quantitative and qualitative. It's my most downloaded plugin, currently sitting at 4,180 downloads, as well as my highest grossing one. It's also the plugin that has rendered me the most positive interactions on social media, with people from all over the world writing me comments on how useful they found it, small audio blogs featuring my work with a write up, and strangers in forums and discord servers recommending *Vendaval* to other people. Of all these interactions, the one that stayed with me the most was a message I got from a sound designer saying they used *Vendaval* to make sound for a virtual reality project that won and award from the Australian Screen Sound Guild.

6 Conclusions

I have found developing my own plugins to be rewarding in many ways. Being able to design my own tools feels like an accomplishment in and of itself, using my own tools on my own work makes me feel much more attached and excited about it, and seeing how other people being excited about the tools I developed is a feeling I never thought I'd get to experience before immersing myself with Csound. I am constantly using these plugins in my own work, which have lead me to sonic avenues and aesthetics I would likely never have arrived without them and I am occasionally still updating them whenever I feel like I am missing a feature, always making a point to publish the updated versions.

I have also been experimenting with using these plugins in real-time applications with the CsoundUnity wrapper. Having complex instruments like these, specially *Granulera*, has been helpful to quickly prototype interactions and ideas in Unity.

Ultimately I still don't know where this will lead. Developing these tools has brought me a sense of accomplishment and I will very likely develop more of them, trying my best to focus on making them unique and accessible to the audio community.⁶

⁶ If the paper is accepted, more links to examples and demonstrations will be added in the footnotes and audiovisual examples will be played during the presentation.